Programming Fundamentals Theory

Assignment 3: Outputs and Explanations

Name: Muhammad Farhan
Roll no: 25K-0538
Section: BCS-1K
Course: Programming Fundamentals Theory
Course Instructor: Miss Uzma Raza

Question 1:

Question5.c | Question2.c | Question3.c | Question1.c | Question6.c

```c
#include <stdio.h>
#include <math.h>
void calculateRepayment(float loan,float interestRate,int years);
int main()
{
    int years;
    float loan,interestRate;

    printf("Enter Loan: ");
    scanf("%f", &loan);
    printf("\nEnter Years: ");
    scanf("%d", &years);
    printf("\nEnter Interest Rate percentage: ");
    scanf("%f", &interestRate);
    interestRate/=100;
    calculateRepayment(loan, interestRate, years);
    printf("\n");
    return 0;
}

void calculateRepayment(float loan,float interestRate,int years)
{
    float installment;
    static float totalrepay=0;
    static int i=1;
    //This is the base case, if the years are completed or if loan becomes approximately 0.
    if (years==0 || loan<1)
    {
        printf("\n\nThe total repayment will be %.3f", totalrepay);
        return;
    }
    //The formula or calculation below is called "loan amortization (installment) formula".
    //It is used to calculate the installment you need to pay in such a way that the loan always equals to 0 in the end.
    //Also, as the loan amount each year changes, it actually wont have any effect on the installment formula because the years decrease.
    installment = (loan * interestRate*pow(1+interestRate, years))/(pow(1+interestRate, years)-1);
    totalrepay+=installment;
    //This is the loan calculation.
    loan = (loan + loan*interestRate)-installment;
    printf("\nYear %d: Remaining Loan = %.1f", i, loan);
    ++i;
    //recursion
    calculateRepayment(loan, interestRate, years-1);
}
```

Explanation:

The program is explained by the comments in the code.

The function calls itself (recursion) and each call decreases the year by 1 until the years are over or until the loan is fully paid.

```
Enter Loan: 1000000

Enter Years: 10

Enter Interest Rate percentage: 10%

Year 1: Remaining Loan = 937254.6
Year 2: Remaining Loan = 868234.7
Year 3: Remaining Loan = 792312.8
Year 4: Remaining Loan = 708798.6
Year 5: Remaining Loan = 616933.1
Year 6: Remaining Loan = 515881.1
Year 7: Remaining Loan = 404723.8
Year 8: Remaining Loan = 282450.8
Year 9: Remaining Loan = 147950.5
Year 10: Remaining Loan = 0.0

The total repayment will be 1627453.875
[Finished in 16.65s]
```

Question 2:

```c
#include <stdio.h>
void calculateFuel(int fuel, int consumption, int recharge, int solarBonus, int planet, int totalPlanets);
int main()
{
    //The values werent mentioned in the question, so i have assumed my own.
    int fuel=100, consumption=10, recharge=5, solarBonus=15, planet=1, totalPlanets;
    printf("The journey will start with 100 fuel, with 10 fuel consumption per planet.\nEnter Total number of planets to travel: ");
    scanf("%d", &totalPlanets);
    calculateFuel(fuel, consumption, recharge, solarBonus, planet, totalPlanets);
    printf("\n");
    return 0;
}

void calculateFuel(int fuel, int consumption, int recharge, int solarBonus, int planet, int totalPlanets)
{

    //This is the first base case, if we manged to travel all the planets.
    if(planet>totalPlanets)
    {
        printf("\nThe journey has been successfully completed.");
        return;
    }
    //This is the second base case, if our fuel runs out.
    else if (fuel==0)
    {
        printf("\nThe fuel has been exchausted, journey incomplete.");
        return;
    }
    if (planet%3==0) //No pattern was given in the question, therefore I have assumed the gravitational zones occur every 3 planets.
    {
        printf("\nExtra 5 fuel renegerated due to gravitational assists.");
        fuel+=recharge;
    }
    else if (planet%4==0)
    {
        printf("\nExtra 15 fuel renegerated due to Solar.");
        fuel+=solarBonus;
    }
    fuel-=consumption;
    printf("\nPlanet %d: Fuel remaining = %d", planet, fuel);
    calculateFuel(fuel, consumption, recharge, solarBonus, planet+1, totalPlanets);
}
```

```
The journey will start with 100 fuel, with 10 fuel consumption per planet.
Enter Total number of planets to travel: 10

Planet 1: Fuel remaining = 90
Planet 2: Fuel remaining = 80
Extra 5 fuel renegerated due to gravitational assists.
Planet 3: Fuel remaining = 75
Extra 15 fuel renegerated due to Solar.
Planet 4: Fuel remaining = 80
Planet 5: Fuel remaining = 70
Extra 5 fuel renegerated due to gravitational assists.
Planet 6: Fuel remaining = 65
Planet 7: Fuel remaining = 55
Extra 15 fuel renegerated due to Solar.
Planet 8: Fuel remaining = 60
Extra 5 fuel renegerated due to gravitational assists.
Planet 9: Fuel remaining = 55
Planet 10: Fuel remaining = 45
The journey has been successfully completed.
[Finished in 5.06s]
```

```
The journey will start with 100 fuel, with 10 fuel consumption per planet.
Enter Total number of planets to travel: 21

Planet 1: Fuel remaining = 90
Planet 2: Fuel remaining = 80
Extra 5 fuel renegerated due to gravitational assists.
Planet 3: Fuel remaining = 75
Extra 15 fuel renegerated due to Solar.
Planet 4: Fuel remaining = 80
Planet 5: Fuel remaining = 70
Extra 5 fuel renegerated due to gravitational assists.
Planet 6: Fuel remaining = 65
Planet 7: Fuel remaining = 55
Extra 15 fuel renegerated due to Solar.
Planet 8: Fuel remaining = 60
Extra 5 fuel renegerated due to gravitational assists.
Planet 9: Fuel remaining = 55
Planet 10: Fuel remaining = 45
Planet 11: Fuel remaining = 35
Extra 5 fuel renegerated due to gravitational assists.
Planet 12: Fuel remaining = 30
Planet 13: Fuel remaining = 20
Planet 14: Fuel remaining = 10
Extra 5 fuel renegerated due to gravitational assists.
Planet 15: Fuel remaining = 5
Extra 15 fuel renegerated due to Solar.
Planet 16: Fuel remaining = 10
Planet 17: Fuel remaining = 0
The fuel has been exchausted, journey incomplete.
[Finished in 2.80s]
```

Explanation:

The code is explained by the comments. The function calls itself with increasing count of current number of planets traveled. The recursion ends when we have traveled all planets or the fuel has ran out.

Question 3:

(The programs now are too large to include screenshots of, it would take 4 screenshots or more, so I will just be putting the output here and explaining the code if necessary (the code already has a lot of comments explaining it), The code is included in the .zip file.)

```
Enter the number of employees to add: 3

Enter Employee#1 ID: 123

Enter Employee#1 Name: Farhan

Enter Employee#1 Designation: Cyber Security

Enter Employee#1 salary: 120000

Enter Employee#2 ID: 234

Enter Employee#2 Name: Majid

Enter Employee#2 Designation: Software Engineer

Enter Employee#2 salary: 110000

Enter Employee#3 ID: 345

Enter Employee#3 Name: Zohaib

Enter Employee#3 Designation: Data Analyist

Enter Employee#3 salary: 90000


1. Display all employees.
2. Find highest salary.
3. Search employee by ID or Name.
4. Exit program.
Enter your choice: 1

Employee#       Employee ID     Name            Designation         Salary
1               123             Farhan          Cyber Security          120000
2               234             Majid           Software Engineer               110000
3               345             Zohaib          Data Analyist           90000

1. Display all employees.
2. Find highest salary.
3. Search employee by ID or Name.
4. Exit program.
Enter your choice: 2
```

```
Enter your choice: 2


The Employee with Highest salary is:
Employee#1 ID:          123
Enter Employee#1 Name:          Farhan
Enter Employee#1 Designation:           Cyber Security
Enter Employee#1 salary:                120000

1. Display all employees.
2. Find highest salary.
3. Search employee by ID or Name.
4. Exit program.
Enter your choice: 3


1. Search by ID.
2. Search by Name
3. Return to main menu.
Enter your choice: 1

Enter exact Employee ID: 234

Employee#2 ID:          234
Enter Employee#2 Name:          Majid
Enter Employee#2 Designation:           Software Engineer
Enter Employee#2 salary:                110000
1. Search by ID.
2. Search by Name
3. Return to main menu.
Enter your choice: 2

Enter exact Employee Name: Zohaib

Employee#2 ID:          345
Enter Employee#2 Name:          Zohaib
Enter Employee#2 Designation:           Data Analyist
Enter Employee#2 salary:                90000
1. Search by ID.
2. Search by Name
3. Return to main menu.
Enter your choice: 3
```

```
Enter exact Employee Name: Zohaib

Employee#2 ID:          345
Enter Employee#2 Name:          Zohaib
Enter Employee#2 Designation:           Data Analyist
Enter Employee#2 salary:                90000
1. Search by ID.
2. Search by Name
3. Return to main menu.
Enter your choice: 3


1. Display all employees.
2. Find highest salary.
3. Search employee by ID or Name.
4. Exit program.
Enter your choice: 4
[Finished in 148.98s]
```

Explanation:

The entire code is basically explained in the comments of the code, so I request the reader to check them. Theres not much else to explain here in this PDF.
Here I will explain the last part of the question: *"If the company later wants to give a 10% salary bonus to employees whose salary is below a certain threshold (say, 50,000), Explain how you could modify your existing program using a function with structure arrays passed by reference to update salaries directly."*

If the company later wants to give a 10% salary bonus to employees whose salary is below a certain threshold (say, 50,000), Explain how you could modify your existing program using a function with structure arrays passed by reference to update salaries directly.

To do this, I would create a new function for this, void updateSalary(struct employeeData employees[], int n).

use a if condition inside a loop to go through each employee whose salary is lower than, say 50000.

then if a employee's salary is lower than 50000, i will use employee[i].salary += (employee[i].salary)*0.10

Here we are not using pointers because, the array by default returns a address and works like a pointer, unlike a normal variable.

This will update the salaries directly without having to return anything.

Question 4:

```
Enter the capacity of the shelf and number of input commands (separated by a space):
3
7




Input           Output
ADD 123 8
ADD 234 9
ADD 345 4
ADD 456 6
ACCESS 123
                -1
ACCESS 456
                6
ACCESS 234
                9
[Finished in 106.50s]
```

Explanation:

Again, the entire code is explained in the comments inside the code, so I request the reader to check the code. Although one thing, I couldn't make the input line up exactly with the output, because you *have* to press *Enter* in order to give the input, which puts a newline by default. So…the output line will always be 1 line lower than the input line.

Question 5:

```
FAST University Text Editor

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 1
Enter index (0 to 0): 0
Enter text: Hello, I hope you are having a great day
Line inserted at 0

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 1
Enter index (0 to 1): 1
Enter text: Finals are near...
Line inserted at 1

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 1
Enter index (0 to 2): 2
Enter text: Theres a lot of pressure, and the exams are gonna be tough, this is pain
Memory expanded: 4
Line inserted at 2
```

```
Memory expanded: 4
Line inserted at 2

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 3

Content:
Lines: 3, Capacity: 4
0: Hello, I hope you are having a great day
1: Finals are near...
2: Theres a lot of pressure, and the exams are gonna be tough, this is pain

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 2
Enter index to delete (0 to 2): 2
Line deleted at 2

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 4
Memory optimized: 2
```

```
Enter your choice: 4
Memory optimized: 2

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 3

Content:
Lines: 2, Capacity: 2
0: Hello, I hope you are having a great day
1: Finals are near...

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 5
Enter filename: Testing
Saved 2 lines to Testing

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 6
Enter filename: Testing
Memory freed
Buffer initialized: 2
Line inserted at 0
Line inserted at 1
Loaded 2 lines from Testing
```

```
Loaded 2 lines from Testing

Menu
1. Insert Line
2. Delete Line
3. Print All Lines
4. Shrink Memory
5. Save to File
6. Load from File
7. Exit
Enter your choice: 7
Memory freed
Goodbye.
[Finished in 94.50s]
```

Explanation:

Again, there are many comments inside the code that explain a lot of what the code is doing. But ill explain it here as a paragraph as well.

So this program creates a dynamic text editor by treating the document as a flexible list of strings instead of a fixed grid like arrays do. The main part of the system is the SheetData structure, which manages an array of pointers (lines), where each pointer corresponds to a single row of text. This design is efficient because it allows the program to use memory only for the lines that actually exist, and each line takes up exactly as many bytes as it needs.

Inside the functions, the logic relies heavily on standard memory management tools. For instance, insertLine first checks if the array is full; if it is, it uses realloc to double the capacity. Once there is enough space, memmove shifts the existing pointers down to create a specific gap at the target index. The function then uses malloc to allocate the exact amount of memory needed for the new string and copies the text into that slot. In contrast, deleteLine starts by freeing the memory of the target string to prevent leaks, and then it uses memmove to shift the remaining pointers up, closing the gap in the array.

To manage data storage, the saveFile and loadFile functions handle the transfer between RAM and the hard drive. saveFile is simple, it opens a file in write mode and goes through the lines array, writing each string to the file followed by a newline character. loadFile is a bit more complex, it first calls freeSheet to clear any existing data in memory to avoid mixing documents. It then reads the file line by line into a temporary buffer. Importantly, it reuses the insertLine function for each line read from the file, allowing the program to automatically handle all the memory allocation and array resizing just like the user was typing the lines in manually.

Question 6:

```
Loaded 0 records from members.dat.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 1
Enter ID: 123
Full Name: Farhan
Batch (CS/SE/Cyber/AI): Cyber
Membership Type (IEEE/ACM): ACM
Reg Date (YYYY-MM-DD): 2024-11-28
DOB (YYYY-MM-DD): 2006-9-28
Interest (IEEE/ACM/Both): ACM
Student added successfully.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 1
Enter ID: 234
Full Name: Mohsin
Batch (CS/SE/Cyber/AI): AI
Membership Type (IEEE/ACM): IEEE
Reg Date (YYYY-MM-DD): 2024-1-20
DOB (YYYY-MM-DD): 2005-5-16
Interest (IEEE/ACM/Both): Both
Student added successfully.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 1
Enter ID: 345
Full Name: Majid
```

```
Enter ID: 345
Full Name: Majid
Batch (CS/SE/Cyber/AI): CS
Membership Type (IEEE/ACM): ACM
Reg Date (YYYY-MM-DD): 2025-7-26
DOB (YYYY-MM-DD): 2007-7=19
Interest (IEEE/ACM/Both): IEEE
Student added successfully.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 2
Enter Student ID to update: 345
Updating Record for Majid...
New Batch (current: CS): SE
New Membership Type (current: ACM): IEEE
Database synchronized with file.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 4

ID    Name                 Batch      Type       Interest
123   Farhan               Cyber      ACM        ACM
234   Mohsin               AI         IEEE       Both
345   Majid                SE         IEEE       IEEE
```

```
ID    Name                Batch      Type       Interest
123   Farhan              Cyber      ACM        ACM
234   Mohsin              AI         IEEE       Both
345   Majid               SE         IEEE       IEEE

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 5
Enter Batch to filter (CS/SE/Cyber/AI): Cyber

 Report for Batch: Cyber
123   Farhan              Cyber      ACM        ACM

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 3
Enter Student ID to delete: 345
Database synchronized with file.
Student deleted successfully.

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 4

ID    Name                Batch      Type       Interest
123   Farhan              Cyber      ACM        ACM
234   Mohsin              AI         IEEE       Both
```

```
ID    Name                Batch      Type       Interest
123   Farhan              Cyber      ACM        ACM
234   Mohsin              AI         IEEE       Both

Menu
1. Register New Student
2. Update Membership/Batch
3. Delete Registration
4. View All Registrations
5. Generate Batch-wise Report
6. Exit
Select option: 6
Database synchronized with file.
Memory freed
Database saved. Exiting...
[Finished in 380.22s]
```

Explanation:

Again, there are a lot of comments in the code explaining the things as well alongside here.

So, this program works as a student registration manager that uses a dynamic array to handle records effectively. The system's main part is the Database structure, which doesn't store data directly but manages a pointer to the Student array. This setup allows the program to allocate memory on the heap using malloc and expand the list as needed, rather than being limited to a fixed size of 100 like in an array.

Inside the functions, the memory management happens automatically. For example, addStudent first checks if the current array is full. If it is, it uses realloc to double the capacity, ensuring there is always room for new students. When removing a student, deleteStudent finds the specific ID and uses a loop to shift all the following records one position to the left. This action effectively overwrites the deleted entry and fills the gap in the list without leaving empty spaces.

To ensure data is saved between sessions, the saveDatabase and loadDatabase functions handle binary file input and output. loadDatabase runs at the start of the program, reading the raw struct data from members.dat directly into the allocated memory to restore the previous session. saveDatabase is called whenever a change occurs, like adding, updating, or removing a student, to update the file with the current contents of memory, keeping the file and RAM in sync.